Adversarial Search in Isolation: Minimax, Alpha-Beta Pruning & Monte Carlo Tree Search

Amr Yasser

December 14, 2025

Abstract

This project implements the game *Isolation* and compares three adversarial search algorithms: Minimax, Alpha-Beta pruning, and Monte Carlo Tree Search (MCTS). We evaluate them under controlled conditions (same depth), under realistic constraints (same time budget via iterative deepening), and in gameplay against a random baseline. We also demonstrate a critical practical insight: Alpha-Beta pruning effectiveness depends strongly on move ordering. Additionally, we compare MCTS—a simulation-based approach—against the traditional tree-search methods. Results show that Alpha-Beta expands far fewer nodes than Minimax at the same depth, while MCTS provides an alternative approach that doesn't require evaluation functions.

1 Introduction

Adversarial search is a core technique for decision-making in deterministic, turn-based, perfect-information games. Minimax provides an optimal decision rule (under a depth limit and heuristic evaluation). Alpha-Beta pruning is an optimization that returns the same move as Minimax at a fixed depth, but can avoid exploring large parts of the game tree. Monte Carlo Tree Search (MCTS) takes a fundamentally different approach, using random simulations to estimate move quality without requiring a hand-crafted evaluation function.

This report covers:

- the Isolation game, including state representation, actions, and transition function,
- Minimax, Alpha-Beta, and MCTS algorithms in simple terms,
- the evaluation function used (for Minimax/Alpha-Beta),
- experimental results: time, nodes explored, depth reached, and simulations,
- comparison of tree-search vs simulation-based approaches,
- reflections on observed behavior and efficiency.

2 Game: Isolation

2.1 State representation

A state s stores:

- a board grid where each cell is empty, blocked, or occupied by A or B,
- the current position of each player (or `None` before initial placement),
- the active player (whose turn it is).

2.2 Initial state

The initial state is an empty board (no blocks) with both player positions unset. The first time each player moves, they **place** their piece on any empty cell.

2.3 Actions (legal moves)

Let the active player be p with current position (r, c).

- If p has not been placed yet, the action set is all empty cells.
- Otherwise, an action is a **queen move**: slide any number of squares in one of 8 directions (horizontal/vertical/diagonal) until blocked/occupied.

2.4 Transition function

Given a state s and action a (a destination cell), the transition $T(s, a) = s'$ is:

- move the active player to a,
- mark the previous player cell as blocked (unusable),
- switch the active player.

2.5 Terminal states

A state is terminal if the active player has **no legal actions**. The player to move loses; equivalently, the winner is the opponent of the active player in the terminal state.

2.6 Game Tree Complexity

- **Branching factor**: Initially 36 (all cells available), averaging around 10–15 in mid-game as cells become blocked.
- **Depth**: Games typically last 20–35 moves depending on play style.
- **Suitability**: The moderate branching factor and depth make Isolation ideal for comparing search algorithms—deep enough to be non-trivial, but manageable for real-time play.

3 Algorithms

3.1 Minimax (depth-limited)

Minimax searches a game tree to depth d, alternating:

- **max** levels: choose the move that maximizes utility for the root player,
- **min** levels: choose the move that minimizes utility (opponent response).

At depth 0 or terminal states, a heuristic evaluation estimates utility.

3.2 Alpha-Beta pruning

Alpha-Beta pruning returns the same move/value as Minimax at a fixed depth, but maintains:

- α: best value found so far for the maximizing player,
- β: best value found so far for the minimizing player.

If $\alpha \geq \beta$, the remaining sibling branches cannot affect the final choice and are pruned.

3.3 Monte Carlo Tree Search (MCTS)

MCTS is a simulation-based algorithm that learns move quality through random playouts. It does not require an evaluation function. Each iteration consists of four phases:

1. **Selection**: Starting from the root, repeatedly select the best child using UCB1 until reaching a leaf.
2. **Expansion**: Add a new child node for an unexplored move.
3. **Simulation**: Play random moves from the new node until a terminal state is reached.
4. **Backpropagation**: Update win/visit counts from the terminal result back to the root.

The UCB1 formula balances exploitation and exploration:

$$\mathrm{UCB1}(v) = \frac{w_v}{n_v} + c\sqrt{\frac{\ln N_p}{n_v}}$$

where w_v is wins, n_v is visits for node v, N_p is parent visits, and $c \approx 1.41$ is the exploration constant.

3.4 Evaluation function (for Minimax/Alpha-Beta)

We use a standard Isolation heuristic: **mobility difference**

$$h(s) = \#\text{legal moves for root} - \#\text{legal moves for opponent}.$$

This is suitable for Isolation because the game is fundamentally about restricting the opponent's mobility. Terminal states return large-magnitude utilities so that forced wins/losses dominate heuristic estimates.

3.5 Iterative deepening and "depth reached"

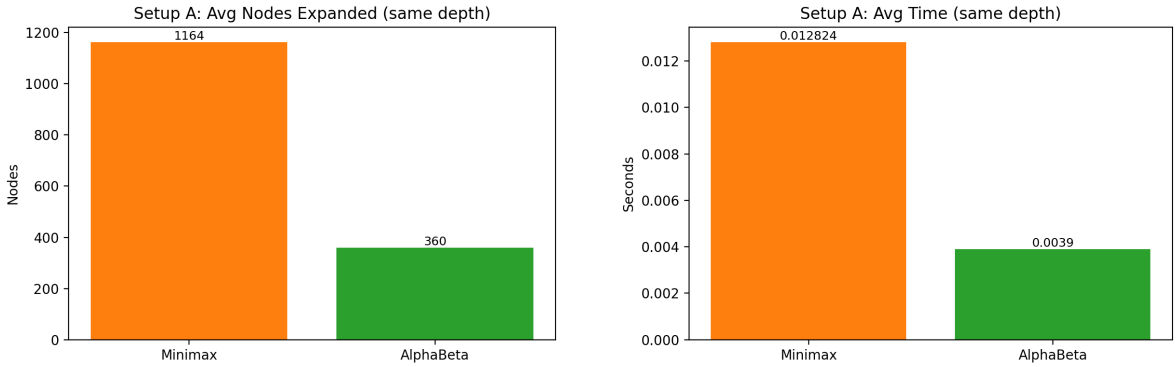
Under a per-move time budget, the agents use iterative deepening: search depth $1, 2, \ldots$ until time runs out. We record **depth reached** as the deepest fully-completed depth within the budget for that move.

4 Experimental Methodology

- **Setup A (same depth):** compare Minimax vs Alpha-Beta fairly at identical depth on identical states.
- **Setup B (same time):** compare behavior under a fixed time budget and record depth reached.
- **Setup C (vs random):** measure win rate and efficiency in gameplay against a random baseline.
- **Setup D (move ordering):** isolate the impact of move ordering on Alpha-Beta pruning.
- **Setup E (MCTS comparison):** compare MCTS against Alpha-Beta under the same time budget.

5 Results

5.1 Setup A (same depth): nodes and time



(a) Average nodes expanded (same depth)

(b) Average runtime (same depth)

Figure 1: Setup A: Alpha-Beta expands fewer nodes and runs faster than Minimax at the same depth.

5.2 Setup B (same time): depth reached

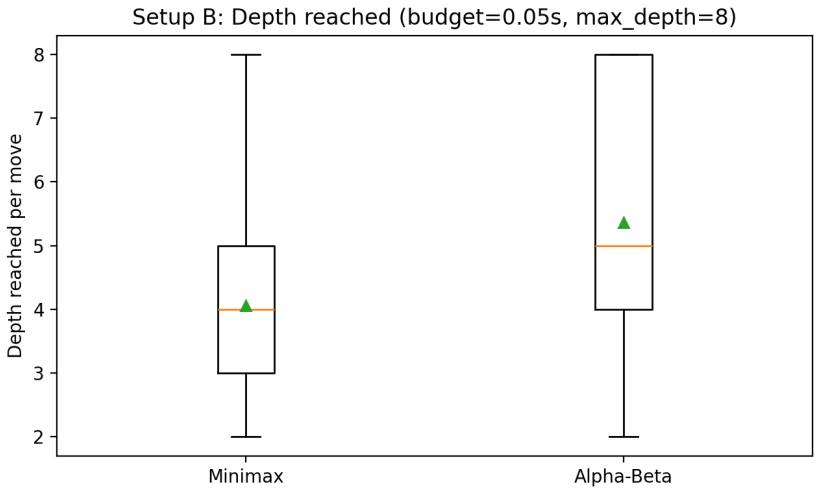
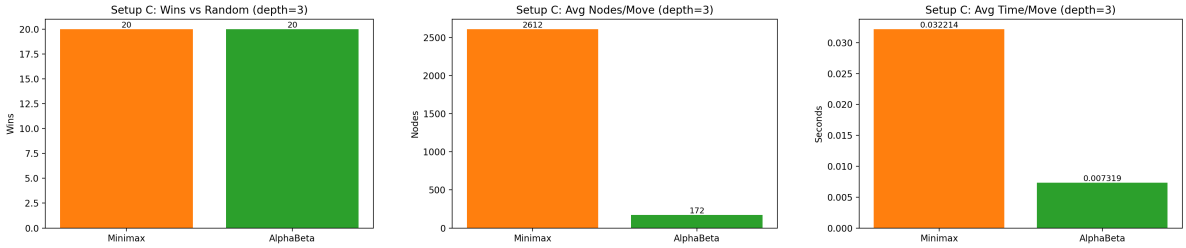


Figure 2: Setup B: Depth reached per move under the same time budget using iterative deepening (mean shown).

This directly quantifies how pruning translates into deeper search under a time constraint.

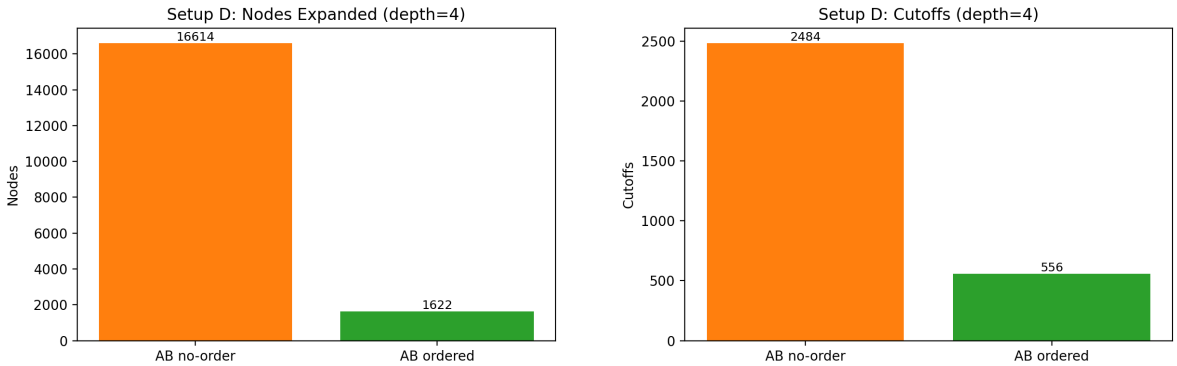
5.3 Setup C (fixed depth vs random): wins, nodes/move, time/move



(a) Win rate vs random (b) Avg. nodes per move (c) Avg. time per move

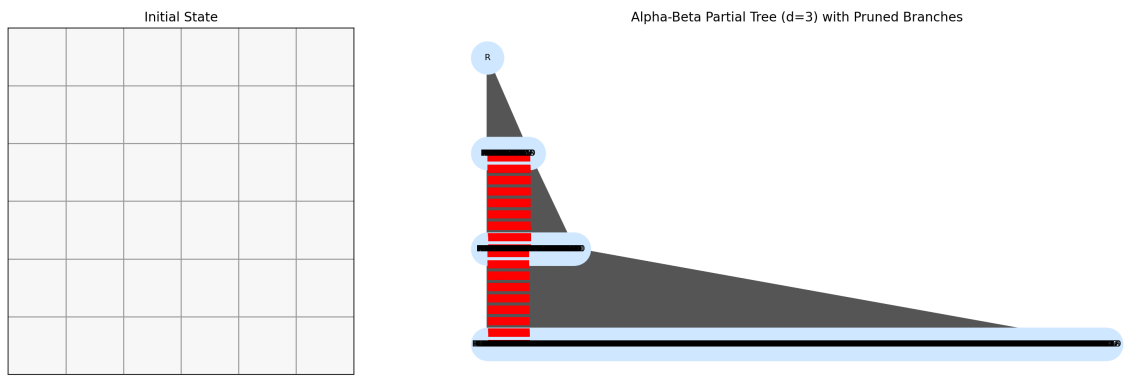
Figure 3: Setup C: Against random play, both search agents win reliably, while Alpha-Beta is significantly more efficient.

5.4 Setup D (move ordering impact): nodes, cutoffs, and pruned tree



(a) Nodes expanded (b) Cutoffs

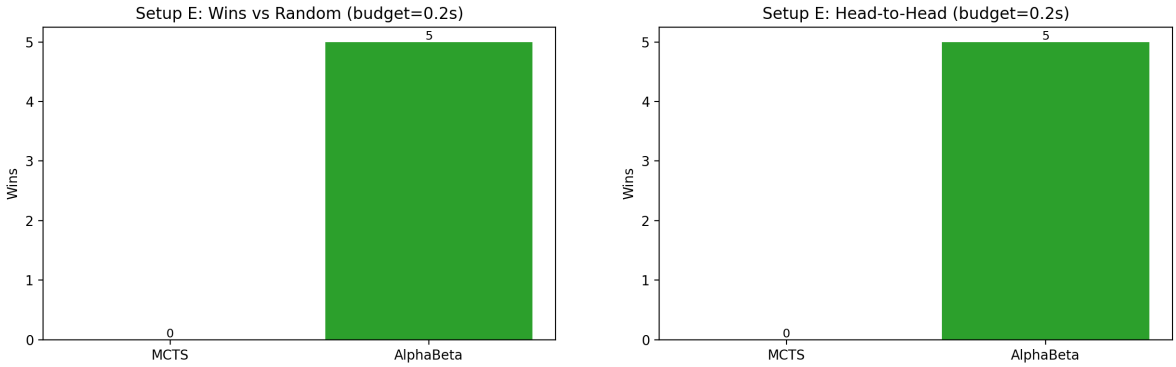
Figure 4: Setup D: Move ordering significantly increases Alpha-Beta pruning effectiveness.



(a) Example start state (b) Partial search tree with pruned branches

Figure 5: Visualization of Alpha-Beta pruning (ordered). Pruned branches are omitted/marked.

5.5 Setup E (MCTS vs Alpha-Beta)



(a) Wins vs Random baseline

(b) Head-to-head matchups

Figure 6: Setup E: MCTS vs Alpha-Beta under the same time budget. Both algorithms perform well against Random, while Alpha-Beta tends to have an edge in head-to-head play for this game.

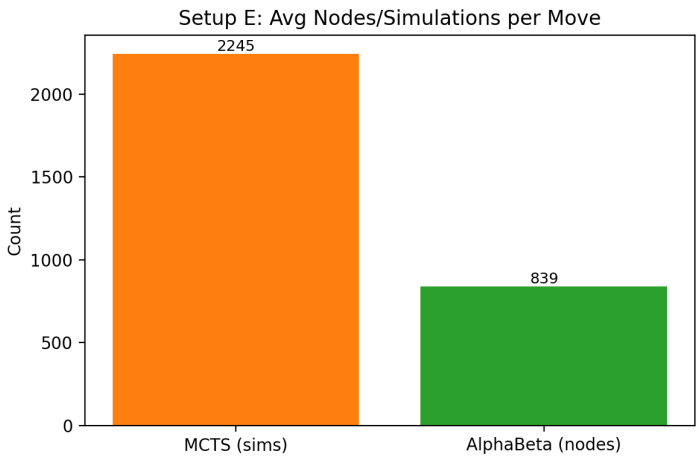
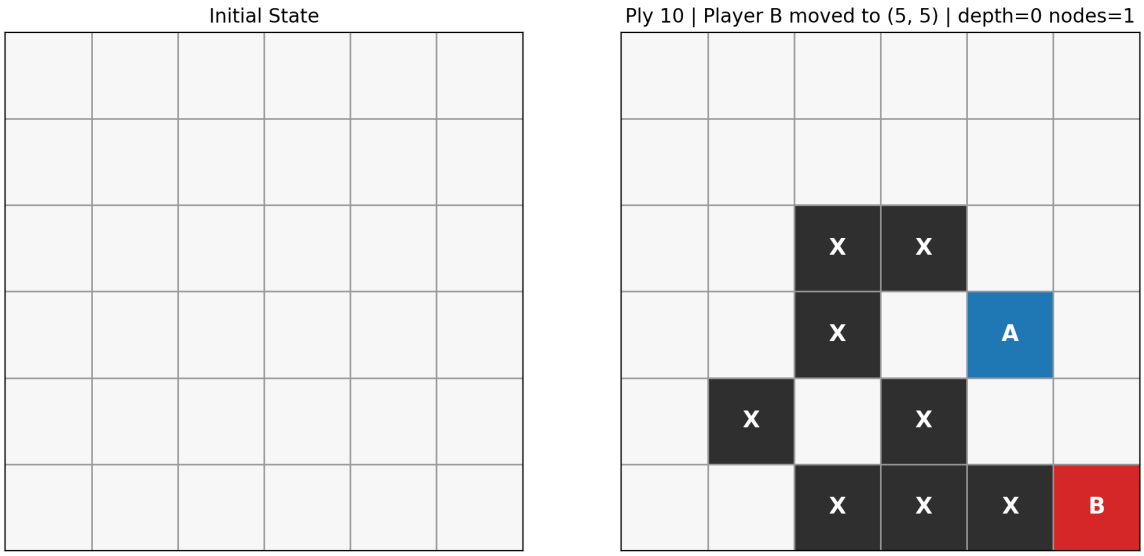


Figure 7: Setup E: Average simulations (MCTS) vs nodes expanded (Alpha-Beta) per move.

6 Qualitative Gameplay Snapshots



(a) Initial placement phase

(b) Later state after several plies

Figure 8: Two snapshots from Alpha-Beta vs Random. Blocked squares accumulate as pieces move.

7 Reflections and Observations

- **Efficiency vs strength:** At fixed depth, Alpha-Beta mainly improves efficiency (fewer nodes/time). Under time limits, the same efficiency often converts to **greater depth reached** (Fig. 2), which can improve playing strength against non-trivial opponents.
- **Move ordering matters:** Alpha-Beta pruning can range from modest to dramatic depending on move ordering, as shown by node and cutoff differences in Setup D.
- **Heuristic suitability:** Mobility difference is a natural fit for Isolation because the win condition is directly tied to restricting legal moves. It is simple, fast, and effective for depth-limited search.
- **MCTS trade-offs:** MCTS requires no evaluation function design—a significant advantage for complex games. However, for Isolation with its clear mobility heuristic, Alpha-Beta with good move ordering tends to outperform MCTS under comparable time budgets. MCTS may shine more in games where good evaluation functions are hard to design.
- **Simulation vs Search:** MCTS explores the game tree "horizontally" through many simulations, while Alpha-Beta explores "vertically" to greater depths. For Isolation's relatively low branching factor, deep search with pruning proves effective.

8 Interactive GUI

In addition to the command-line experiments, we provide a Pygame-based graphical user interface for interactive play. The GUI supports:

- Human vs AI gameplay (select your moves by clicking)

- AI vs AI demonstrations (watch different algorithms compete)
- Real-time statistics display (nodes, depth, simulations, time per move)
- Selection from all implemented agents (Random, Minimax, Alpha-Beta, MCTS)

To launch the GUI: `python -m gui.game_gui`

9 Reproducibility

From the project root:

- Run tests: `python -m pytest -q`
- Run experiments (A–E): `python -m experiments.run_setups`
- Generate all figures + snapshots: `python -m visualization.auto_generate_figures`
- Launch interactive GUI: `python -m gui.game_gui`